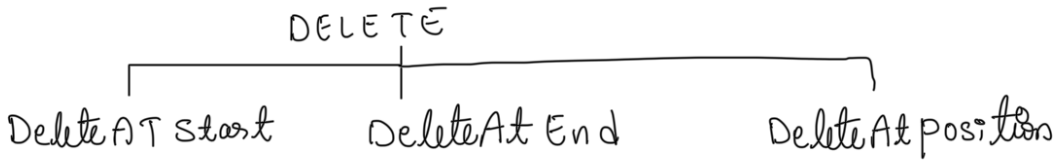
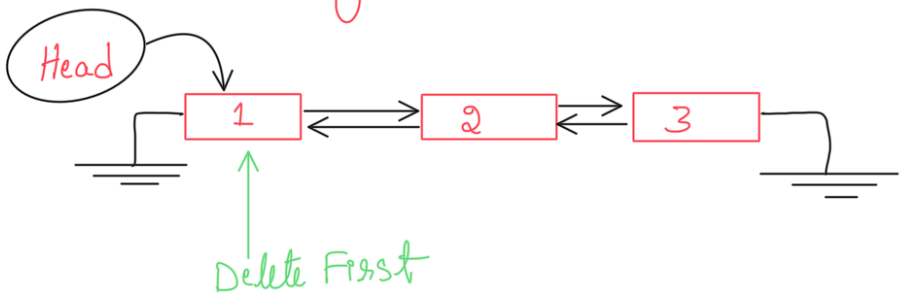
DELETE IN DOUBLY LINKED LIST

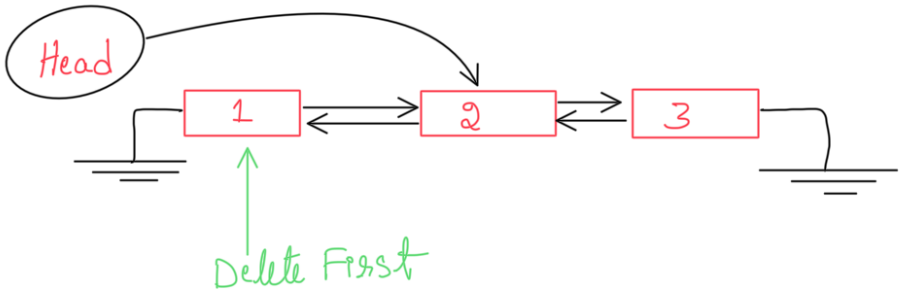
Day 42
21/08/2025



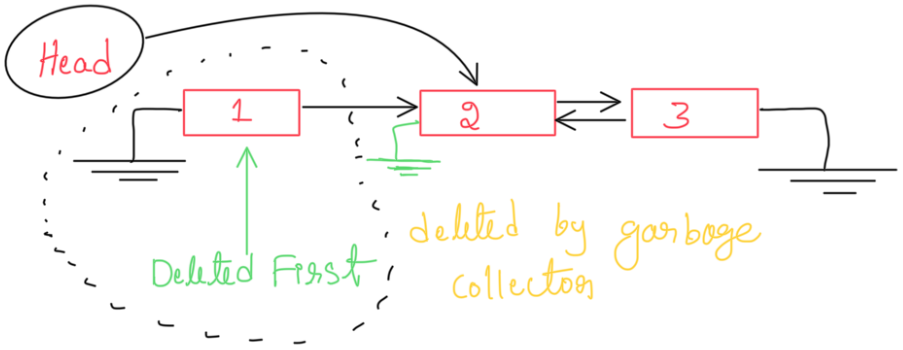
Delete At start (Deleting the HeadNode)

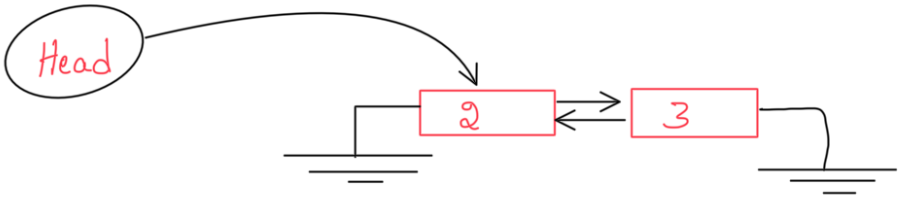


check if (Head.next != null)
shift Head to Head.next



Now Head is updated
and Now also update Head.prev = null





First Node Head Node is Successfully deleted

Step 1
Shifting Head

Step 2
Head.prev = null

Corner case (1)

if List Have Single Node

then only perform step 1
Head = Head.next OR Head = Null

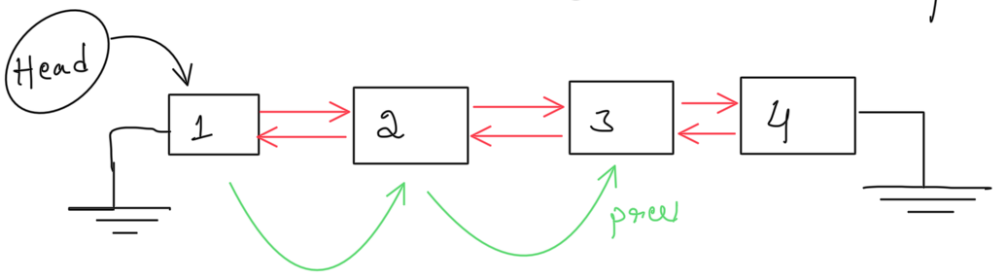
Corner case (0)

if Head is Null
List is Empty So Just return

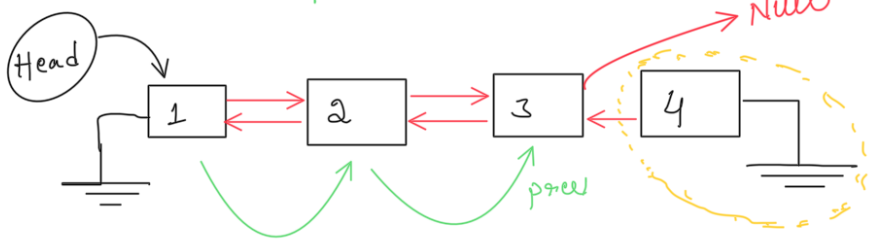
Single Node, Multinode, Empty List	→ Cases While deletion

DELETE AT THE END (Handling Single, Multi, Empty)

Reach the last Second Node & update Links



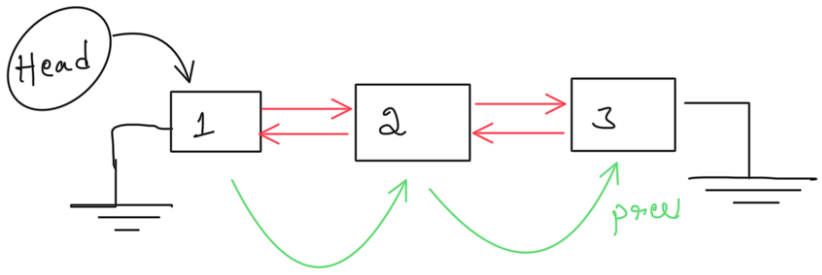
prev.next = Null



Old last node deleted By garbage collector

prev.next = Null

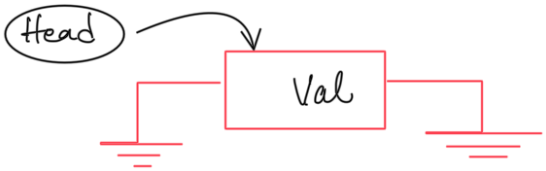
⇩



prev.next = Null

Corner case (1)

The List Has only Single Node

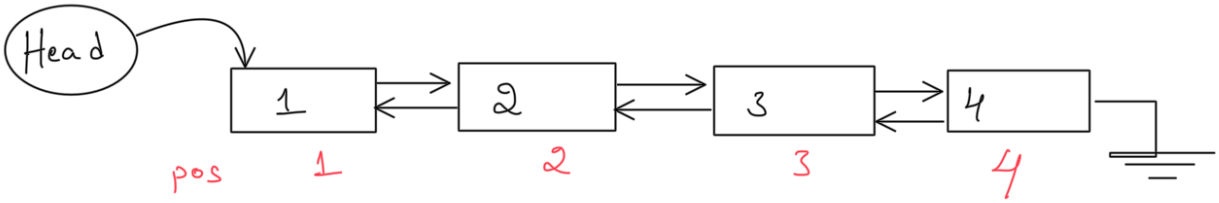


then Just assign Head = Null

Corner case (2)

If Head = Null which means list is Empty So we Just return

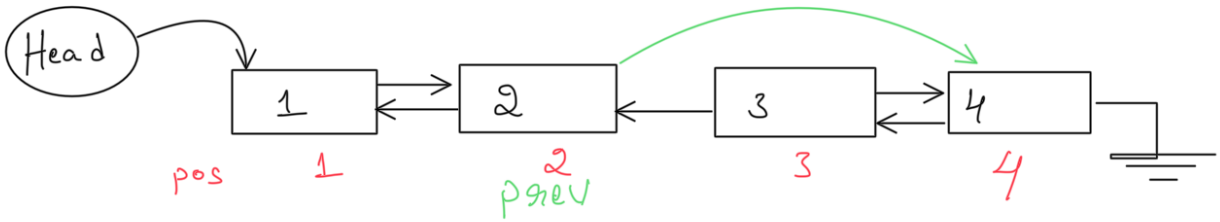
DELETE AT THE position



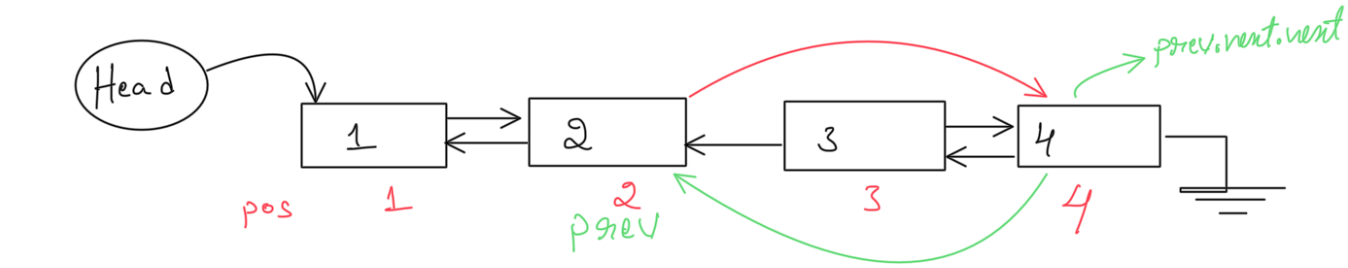
delete AT position 3

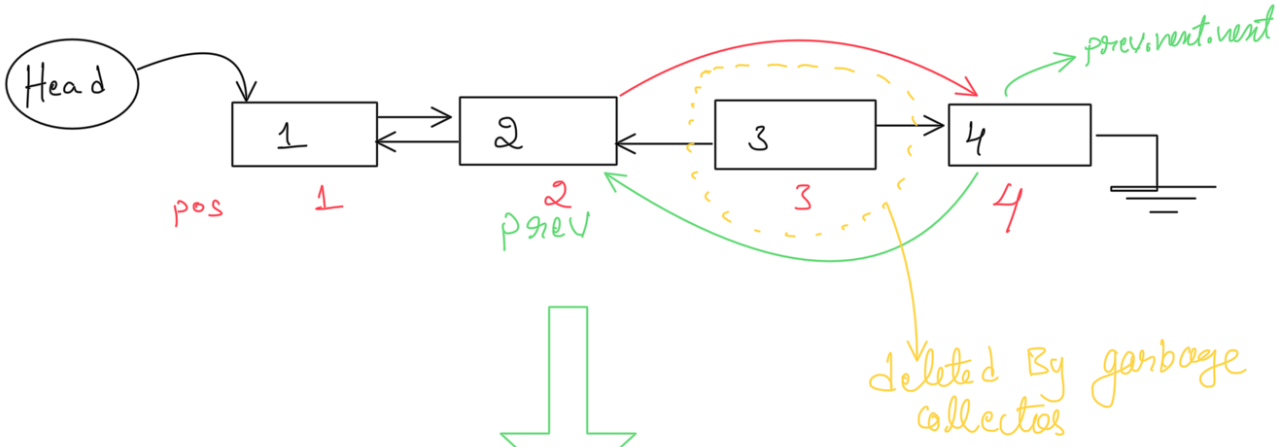
Step 1 Travel till prev node of delete position

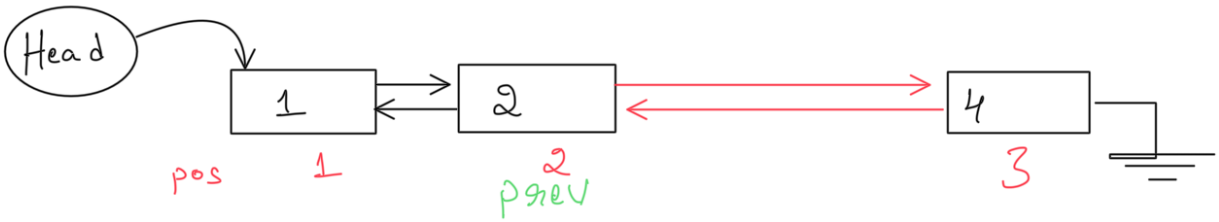
prev.next = prev.next.next;



Step 2 prev.next.prev = prev







Step 1 delNode.next = delNode.next.next

Step 2 delNode.next.prev = delNode

Corner Cases

Case (1) Empty list → return

Case (2) Single Node → if (pos > 1) → return

Case (3) pos == 1 deleteFront() → return

Case (4) pos == size deleteEnd() → return

NOTE : DONT Forgot to update the size of linked list at each Successfull deletion (size--)

Code

```
200
201     // Delete Operations
202
203     // Delete at front
204     public void deleteAtFront(){
205         if( this.head == null ) {
206             System.out.println(x:"The List IS Empty Can't Delete: ");
207             return;
208         }
209
210         if(this.head.next == null){
211
212             this.head =null;
213             this.size--; //Decrement Size ;
214             return;
215         }
216
217         this.head = head.next;
218         this.head.prev = null;
219         this.size--;
220
221     }
222
```

Corner Cases Handled

Size updation

// Size updation

```
222
223     // Delete at End
224     public void deleteAtEnd(){
225
226         if( this.head == null ) {
227             System.out.println(x:"The List IS Empty Can't Delete: ");
228             return;
229         }
230
231         if(this.head.next == null){
232             this.head =null;
233             this.size--; //Decrement Size ;
234             return;
235         }
236
237         Dnode lastNode = head ;
238
239         while ( lastNode .next != null) {
240             lastNode  = lastNode .next;
241         }
242
243         lastNode .prev.next = null;
244         lastNode .next = null ; //optional Clean UP
245         this.size--;
246     }
247
```

Corner case

Size updation

// Size updation

```
248     // DeleteAtPosition
249     public void deleteAtPosition(int position){
250         if( this.head == null ) {
251             System.out.println(x:"The List IS Empty Can't Delete: ");
252             return;
253         }
254
255         if (position == 1) {
256             deleteAtFront();
257             return;
258         }
259
260         if(position == this.size){
261             deleteAtEnd();
262             return;
263         }
264
```

Handling Corner cases

```java
        if(position <= 0 || position > this.size){
            System.out.println(x:"Invalid Position ");
            return;
        }

        int prevPosition = 1;
        Dnode temp = head;

        while (prevPosition < position - 1) {
            temp = temp.next;
            prevPosition++;
        }

        temp.next = temp.next.next;
        temp.next.prev = temp;
        this.size--; // Size updation
    }
```